The aicesat index, from first principles

a functional design for building and querying
sub-granule indexes over many altimetry collections

2026-10-17

Abstract

A scientist asks for every altimetry point of several satellite and airborne missions inside a box on the ice. The data sit in thousands of multi-gigabyte files in a cloud archive, and the box needs a few megabytes of them. This document derives, from that one requirement, a system that reads exactly the bytes the box needs: a *sub-granule index* that records where every spatial cell's points live inside every file, a *build* pipeline that produces the index once, a *query* pipeline that turns a box into byte ranges and a cache of decoded points, and a *framework* in which each collection is a value plugged into one shared pipeline. It is written for a reader who has not seen the current code. Every mechanism is motivated before it is named, every value and stage is drawn in one visual grammar, and every number is a measurement from the deployed instance. The design is partly built: the margin says, chapter by chapter, which pieces exist today.

How to read this document

The text proceeds in five parts. Part I states the problem and the two kinds of data involved: the archive's files and the grid of cells that addresses the Earth. Part II derives the index as a table and fixes what a row means. Part III builds the index and Part IV answers a query from it; both parts separate the *structural* view (which values and functions exist) from the *runtime* view (which processes and threads run them) and the *computation* (what work is done per point, per chunk, per granule). Part V is the framework: how six collections share one pipeline, how configuration and the grid become parameters, how the design is enforced in a language that enforces nothing.

Figures use one grammar throughout (Figure 1). The margin carries two kinds of note: numbered asides, and unnumbered notes in italics saying what already exists in the code for that chapter. Names of modules and functions live in those notes and in the listings, so the body can stay with concepts.

Contents

I The problem and the data **1**

1 What the system must do **2**

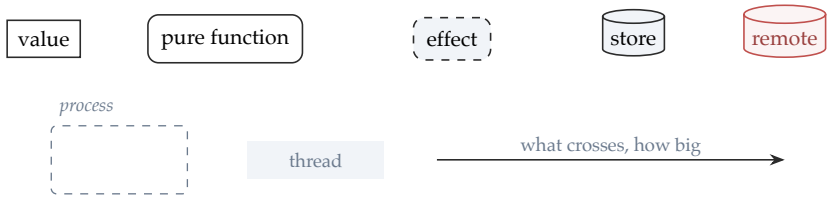


Figure 1: The figure grammar. Sharp corners are data; rounded corners are code; grey dashed code has side effects; cylinders are stores, red when they belong to the archive. Arrows say what crosses and, where measured, how much.

2 What a granule is 3

3 Cells 5

II The index as data 7

4 The index row 7
4.1 The run 7
4.2 Photons: runs with edges 8

5 Files, provenance and the claim 9

III The build pipeline 11

6 Build as a pipeline of values 11

7 Reading one granule 12

8 The build at runtime 14

IV The query pipeline 16

9 Query as a plan 16

10 Executing the plan: one chunk, step by step 17

11 The lake as a memo 19

12 The query at runtime 20

V The framework 22

13 The collection descriptor 22

14 Configuration as a value 24

15 The grid as a parameter 25

16 Replay 26

17 Packages and the structural view 27

18 Purity in Python, and what not to do 28

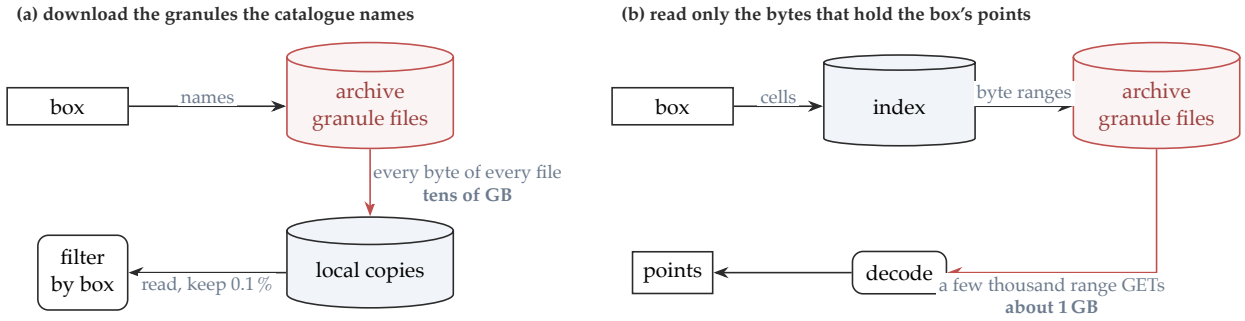


Figure 2: The same box answered two ways. Downloading keeps a tenth of a percent of what it moves. Reading by byte range needs something the archive does not provide: a table saying which bytes of which file hold which part of the Earth. That table is the index.

Appendices **29**

A Migration path **29**

B Test strategy **29**

C Measurements the design leans on **30**

D Glossary **30**

Part I

The problem and the data

1 What the system must do

The request is always the same shape: *every point from these missions, inside this area, in this time window*. The area is a few tens of kilometres across; a glacier, a transect, a study site. The missions are satellite laser altimeters (ICESat's GLAS, ICESat-2's ATL03 photons and ATL06 land-ice segments, GEDI's footprints) and airborne or ground campaigns (IceBridge ATM, a GPS traverse). The answer is a table of positions, heights and times, drawn on a map and fed to analysis.

The data live in the Earthdata cloud: one *granule* per orbit segment or flight, each a file in S3 in the `us-west-2` region, found through a catalogue (CMR) that answers spatial and temporal searches with granule names and URLs. There are two ways to turn a box into points.

Downloading (Figure 2a) is simple and wrong at this scale. An ICESat-2 ATL06 granule is 32 to 128 MB; an ATL03 granule is 0.2 to 6.7 GB, median 1.3 GB over Greenland's Summit; a few hundred granules cross any site, and the box wants a fraction of a percent of each. Reading by byte range (Figure 2b) moves only what the box needs, but S3 can serve a byte range only when the caller already knows the offset and length. Someone must have looked inside every file once and written down where each part of the Earth is.

That table is the **sub-granule index**, and the whole system is organised

around it. It fixes the two costs the design must respect:

- **The build** looks inside every granule once. On the deployed machine the regional indexes hold 33,484 ATL06 granules and 48,838 ATL03 granules; a build is hours, so it must be resumable, parallel and never silently incomplete.
- **The query** must be bounded below by the network and nothing else. In the archive's region a byte range arrives in tens of milliseconds, and a scene's worth of ranges in about three seconds. Everything the query does beyond fetching (decoding, deciding which point belongs to which cell, caching) must cost less than that or it is the bottleneck.

Two more requirements come from use rather than scale. The same area is asked for again and again with small changes, so decoded points are worth keeping in a local *lake* that later queries read first. And the collections differ in file format, chunking and quality semantics while asking the same question, so adding a collection must mean describing it, not re-implementing the pipeline.[1]

[1] Six collections exist today, built and queried by six copies of nearly the same code. Part V is about the seventh.

2 What a granule is

Four of the six collections are HDF5 files; two are CSV text. The design needs only a small part of HDF5, but that part has to be exact, because the index will address bytes inside files it never opens again.

Groups, datasets, chunks. An HDF5 file is a tree of *groups* holding *datasets*. An ATL06 file has six beam groups (`gt1l`, `gt1r`, …, `gt3r`); each holds a group `land_ice_segments` with one dataset per variable: latitude, longitude, height, time, a quality flag. A dataset is stored in *chunks*, fixed-size pieces of the array, each compressed on its own (gzip, usually after a byte-shuffle filter) and written wherever the file had room. The file keeps a B-tree per dataset mapping chunk number to (offset, size). Reading element i means finding chunk $i \operatorname{div} C$ in that tree, fetching its bytes, decompressing them, and taking element $i \bmod C$.

The chunk map (Figure 3) is the only thing the query needs from the file's structure. Walking the tree to obtain it is the expensive part of a build; Chapter 7 is about making that walk cheap. Decoding a chunk is the cheap, pure part: inflate, undo the shuffle, reinterpret as the dtype.[2]

[2] Decoding is validated byte-identical against the reference HDF5 library on real chunks; it needs no HDF5 code at query time.

Three layouts. The collections differ in how their datasets are chunked, and that difference decides what a row of the index can mean (Figure 4).

For the ICESat products and GLAS, every dataset of a group is chunked identically (10,000 segments per chunk in ATL06, 100,000 photons in ATL03), so chunk k of the latitude dataset covers the same elements as chunk k of the height dataset, and one row can carry one byte range per dataset. GEDI chunks its datasets differently; the design cuts the element axis at the union of every dataset's chunk boundaries, so each resulting *span* lies inside exactly one chunk of every dataset and the row again carries one range per dataset, plus the offset of the span inside each chunk. The text collections have no chunks at all: the build scans the whole file once and a row is the byte span of the lines that fall in one cell.

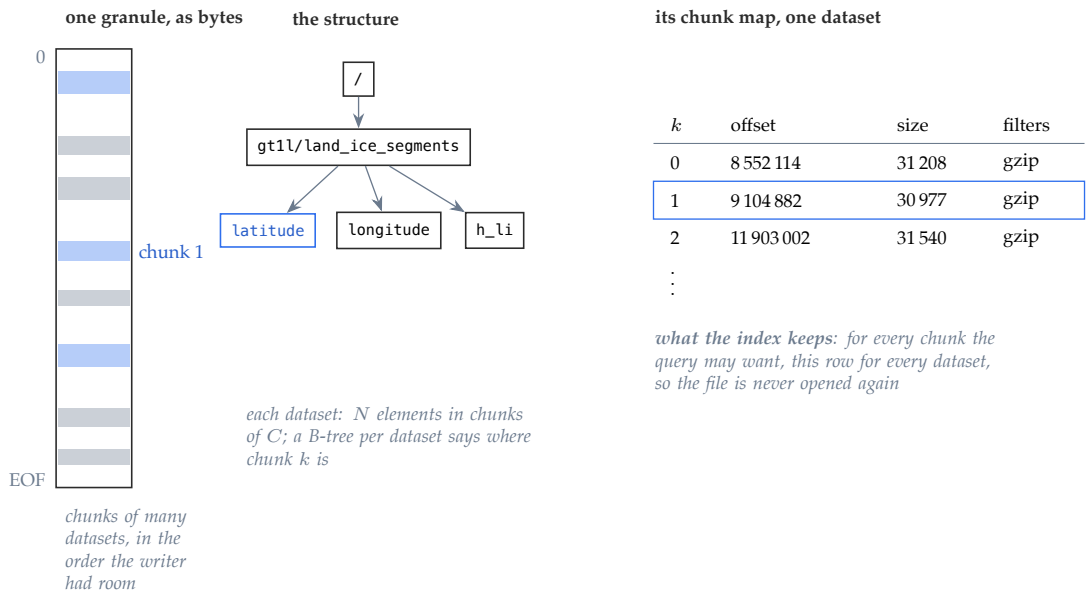


Figure 3: A granule's bytes, its structure, and the chunk map that connects them. The index is the chunk map, reorganised by where on the Earth each chunk's points fall.

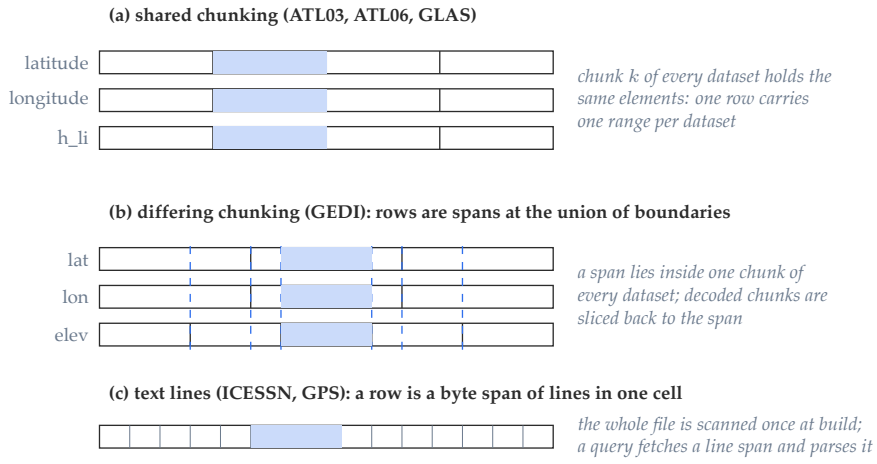


Figure 4: Three storage layouts, one index shape. In each case a row names a contiguous element interval and the byte ranges that hold it, so the query reads ranges and slices without knowing which layout produced the row.

ATL03's indirection. The photon product adds one twist that matters later. Photons are stored by time, and their positions are stored twice: every photon has its own latitude and longitude, and every 20 m *segment* has a reference photon and a count. A build cannot afford to read a billion photon positions, so it reads the segment positions and maps each segment's photon range (`ph_index_beg`, `segment_ph_cnt`) onto chunks. A chunk of 100,000 photons is thus located by the few hundred segments that cover it.

The six collections. Table 1 is the whole of what differs; everything else is shared. It is also, almost verbatim, the descriptor of Chapter 13.

collection	product	layout	position source	group structure
ATL03	ICESat-2 photons	shared chunks	segment reference photons	6 beams; strong/weak
ATL06	ICESat-2 land ice	shared chunks	per segment	6 beams; strong/weak
GLAS	ICESat GLAH06	shared chunks	per shot	one group
GEDI	GEDI L2A	spans	per shot	8 beams; full/coverage power
ICESSN	IceBridge ATM L2	text lines	per platelet line	none
GPS	Summit traverse	text lines	per fix line	none; sled geometry from a metadata file

Table 1: What differs between collections. Each difference is a value or a small function in Part V.

3 Cells

The index must answer "which chunks hold points inside this area" without geometry at query time. The standard device is a discrete global grid: partition the Earth into cells with integer identifiers at several resolutions, assign every point its cell once at build time, and let the query ask for cell identifiers. The current grid is H3,[3] whose cells are hexagons with 64-bit identifiers; a resolution-5 cell is 253 km^2 (edge 9.85 km), resolution 6 is 36 km^2.

[3] H3's hexagons have the pleasant property that a straight track meets a cell in one contiguous stretch, which Chapter 4 exploits. Chapter 15 makes the grid a parameter and says exactly which of its properties the design relies on.

A cell plays three roles, at two resolutions (Figure 5):

1. **Addressing.** Every index row carries the cell of its points at the collection's addressing resolution: 5 for the segment and footprint products, 6 for photons, chosen so that a cell is roughly the footprint of one chunk. A query turns its area into the set of addressing cells it overlaps and asks the index for rows with those cells.
2. **Partitioning.** The lake of decoded points is partitioned by the same cell, so a query reads exactly the cells it asked for and eviction has a unit.
3. **Claiming.** A build records the ground it covered as a set of finer cells (the *claim*), compacted up the hierarchy where whole parents are covered. A later query over an area checks that every fine cell of the area has an ancestor in the claim; a later build over a larger area finds the fine cells not yet claimed and builds only those.

Two consequences shape everything downstream. First, a cell is coarse relative to a box, so the index over-selects by design and the query reapplies the exact box predicate to decoded points; the index is for finding bytes, never for answering the box. Second, the per-point assignment of

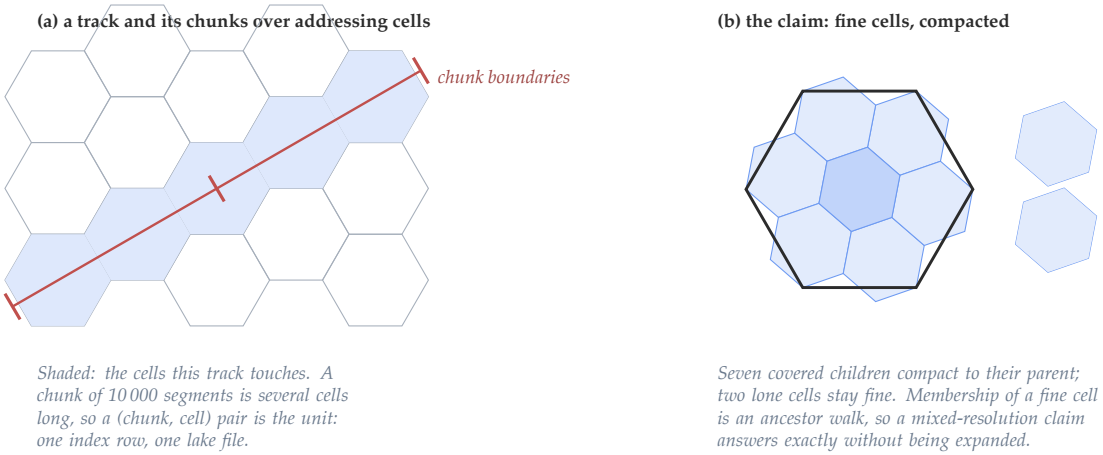


Figure 5: Cells in their two roles. Left, addressing: rows and lake files are keyed by (chunk, cell). Right, claiming: the ground a build covered, stored compactly, consulted by every query and every enlarging build.

a cell is the one operation the system performs *per point* on both sides, build and query, and it is the first thing to remove from the query side (Chapter 4).

what exists today. `planner.cells_for_bbox`, `coverage_cells`, `addressing_cells`; the claim in `index.write_build_manifest` and `unclaimed_cells`; the hot kernel is `planner._cells_vectorized` over h3ronpy.

Part II

The index as data

4 The index row

Everything so far can be assembled into the row the build writes. Take one granule, one group, one chunk k. The build has the positions of the chunk's elements (read once, from the position datasets or the segment table), assigns each its addressing cell, and so knows the set of cells the chunk touches. For each such cell it writes one row. The row has four parts:

- **Identity**: granule name, both URLs (HTTPS and `s3://`), the archive revision, the group (beam) and whether it is a strong beam, orbit numbers, the chunk index and the element interval the chunk covers.
- **The cell**, and the chunk's own bounding box in latitude and longitude. The box is a second, finer filter: a cell reaches kilometres outside a small query box, and the chunk's box prunes rows the cell let through.
- **The run**: where, inside the chunk, this cell's elements sit. Below.
- **Byte ranges**, one per dataset: offset, size, the filter pipeline, dtype, column count and the per-chunk filter mask. This is the chunk map of Figure 3, copied so the file need never be opened again.

Rows are few: a granule contributes about 18 rows for ATL06, 7 for ATL03, 25 for GLAS, 2 for ICESSN, because a chunk is long and a cell is large. The same chunk appears under every cell it touches, with identical byte ranges; the query deduplicates by (granule, group, chunk) before fetching.

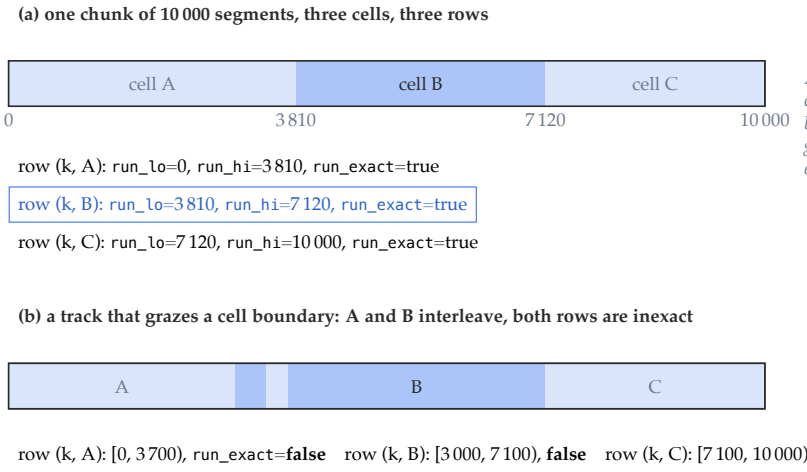
4.1 The run

The build already paid for the one per-point operation: it assigned every positioned element its cell. Without the run, the query would pay it again on every fetched chunk, because after decoding it has 10,000 segments or 100,000 photons and must know which of them belong to the cells it asked for. On the deployed machine that re-assignment was 59 million lookups and 24 to 31 thread-seconds of an 8-cell query.

The run records what the build learned. For a (chunk, cell) row:

$$[\texttt{run_lo},\ \texttt{run_hi}) = \big[\min i,\ \max i + 1\big) \quad \text{over this cell's elements } i,$$
$$\texttt{run_exact} = \big(\#\{i \in [\texttt{lo}, \texttt{hi}) \text{ positioned}\} = \#\{i \text{ in this cell}\}\big).$$

Exactness is decidable at build time because the build has every element's cell. It says: no element of another cell sits inside this interval. When every wanted cell's run is exact, the query labels the chunk's elements by slicing, in constant time per cell, and never calls the grid. A straight-ish track meets a convex cell in one contiguous stretch, so runs are exact unless cells interleave; on real ATL06 data 99.3 % of 9,422 runs over twelve granules are exact, and in an 8-cell ingest all 2,948 chunks were labelled from runs.



An exact run lets the query write the cell into a slice of the label array. An inexact run makes the query fall back to assigning every element of the chunk; it never guesses. Rows from an index built before the columns existed have no run and take the same fallback.

Figure 6: Cell runs. Each row carries the element interval of its cell inside the chunk and whether that interval is pure. The fast path is a slice; the slow path is the per-point assignment, kept alive by every inexact row.

The two pure kernels are short enough to show whole, because they *are* the contract between build and query:

what exists today. `index.cell_runs`, `index.cells_from_runs`; the ATL06, GLAS and GEDI builders write `run_lo/run_hi/run_exact` (schema versions 3, 3, 2); their ingests slice. Counter `chunks_cells_from_runs` in the fetch stats.

```
def cell_runs(pos, cells) -> dict[int, tuple[int, int, bool]]:
    """{cell: (lo, hi, exact)} for one chunk; pos are within-chunk positions of
     its
    positioned points, cells their cells."""
    order = np.argsort(pos, kind="stable"); pos, cells = pos[order], cells[order
     ]
    out = {}
    for cell in np.unique(cells):
        idx = np.flatnonzero(cells == cell)
        lo, hi = int(pos[idx[0]]), int(pos[idx[-1]]) + 1
        inside = np.searchsorted(pos, hi) - np.searchsorted(pos, lo)   #
     positioned points in [lo, hi)
        out[int(cell)] = (lo, hi, inside == idx.size)
    return out

def cells_from_runs(n, runs, wanted) -> np.ndarray | None:
    """Labels for a chunk's n points, or None when the fast path does not apply.
     """
    if not runs: return None
    out = np.zeros(n, "u8")
    for cell in wanted:
        r = runs.get(int(cell))
        if r is None or not r[2]: return None      # missing or inexact: fall
     back
        out[r[0]:r[1]] = int(cell)
    return out
```

4.2 Photons: runs with edges

ATL03 complicates the run in one way, because a photon's cell is known to the build only through its segment. The build assigns a cell to each 20 m segment from its reference photon and expresses the run in photon index space: [`run_lo`, `run_hi`) is the union of the cell's segments' photon ranges inside the chunk. But a segment straddling the cell boundary has photons on both sides, and the index must not claim for a cell photons that belong to its neighbour. So the row records two more numbers, `edge_lo_end` and `edge_hi_start`: where the run's first segment ends and where its last begins (Figure 7). The query labels the interior by slicing and assigns the grid cell individually only to the photons of the two edge segments, and

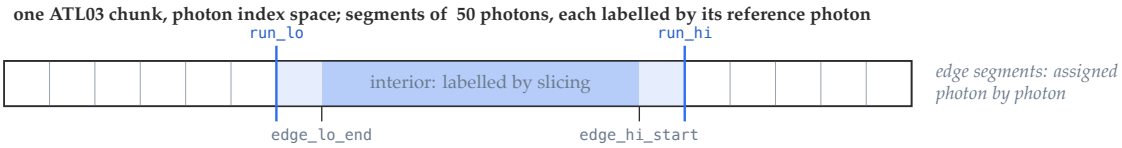


Figure 7: Photon runs. The interior of a cell's run is labelled by slicing; the two edge segments and all photons outside wanted runs are assigned individually, so the result equals assigning every photon.

to photons outside every wanted run. The result is photon-exact, byte-identical to assigning all 100,000, at a few hundred lookups.

what exists today. `index.photon_cell_runs`, `labels_from_photon_runs`; ATL03 schema version 6; the planner's `_materialize` assigns only where `assign` is set and reports `photons_assigned`.

5 Files, provenance and the claim

The rows of one granule are one Parquet file, named after the granule, under the collection's index directory. Writes are to a temporary name followed by a rename, so a reader never sees a torn file and an interrupted build leaves no corpse for the next run to trip on. The file is the unit of resumption: a build lists the directory, and a granule whose file exists with the current schema version is done.

That rule has a hole, and closing it is what the rest of this chapter is about. A build over a region asks for rows only in the cells of that region: a granule's pole-to-pole track would otherwise fill the index with cells no query ever asked for, each holding only the granules that happened to cross the region. So a granule's file is *relative to the cells the build asked for*, and a later build over a larger region cannot tell from the name whether the file holds rows for the added ground.

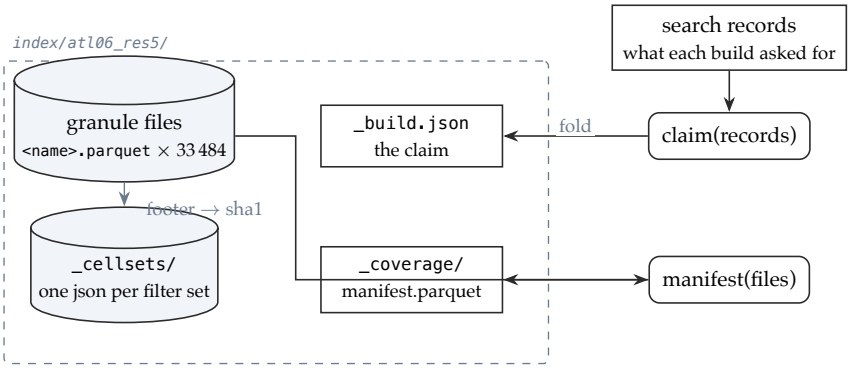
Provenance. Each file's footer therefore records the build's filter set: the compacted cell set the build asked for, stored once as a content-addressed sidecar (`_cellsets/<sha1>.json`) and referenced by its hash.[4] A granule *proves* a set of cells if its recorded set contains them. An enlarging build computes the fine cells not yet claimed and rebuilds exactly the granules that cannot prove them.

[4] The full compacted list was first stamped into every footer; at 49 KB it was three quarters of an ATL06 index file and 3.3 GB of a 7 GB store. The sidecar took the store to 4.7 GB.

The invariant is easy to state and was broken twice: the footer must record the *filter*, never the cells the granule happened to touch. Two builders had shadowed the filter variable with the per-point cell array. For GLAS the recorded set was touched ∩ filter, which was never wrong but gave every granule its own sidecar (4,220 for 4,799 files) and rebuilt everything on every enlargement. For ATL03 it was the unfiltered touched set of the last beam, so a file whose rows had been dropped could prove ground it did not hold, and an enlarging build silently skipped it: the exact class of bug provenance exists to prevent.

what exists today. fixed 2026-10-17; the footer key is `aicesat_index_cellset_sha1`; `index.granule_proves` answers False for unknown provenance; `tests/test_build_provenance.py` pins both builders with a two-cell track and a one-cell filter.

The claim. When every granule of a build has landed, and only then, the build unions its fine cells into the collection's claim (`_build.json`): the compacted set of ground the index covers. Every query checks its area against the claim before doing anything else; an unclaimed area is an error naming the build command, not a slow success through whole-



The claim is derived*: a fold over the records of successful builds. The manifest is* derived*: a fold over the files. Neither is primary, both are cached because their derivation costs time, and both are rebuilt rather than repaired.*

Figure 8: One collection's index on disk. Granule files are primary and immutable; the claim and the manifest are folds over them. The provenance sidecars say which cells each file was built against.

granule fallback. An empty catalogue search is recorded but claims nothing: silence from the catalogue is not evidence of empty ground.

The manifest. Coverage questions (which granules touch these cells, over which months) should not open tens of thousands of footers. The build ends by folding the index into one small manifest of distinct (cell, granule, month) rows (`_coverage/manifest.parquet`), rebuilt whenever the directory's modification time says a file landed. The query uses it to name the handful of files that touch its cells, so a plan reads a few files rather than a glob of 48,838.[5]

What the chapter establishes is a discipline for every derived thing in the system: the granule files are the primary data; the claim and the manifest are folds over them, cached because the fold is expensive, and never edited in place. Chapter 6 turns the build records into a value so the claim becomes a pure function of them.

[5] Before the manifest the ATL03 plan for a 0.3° box scanned every footer: 36 s. Through the manifest, 0.66 s.

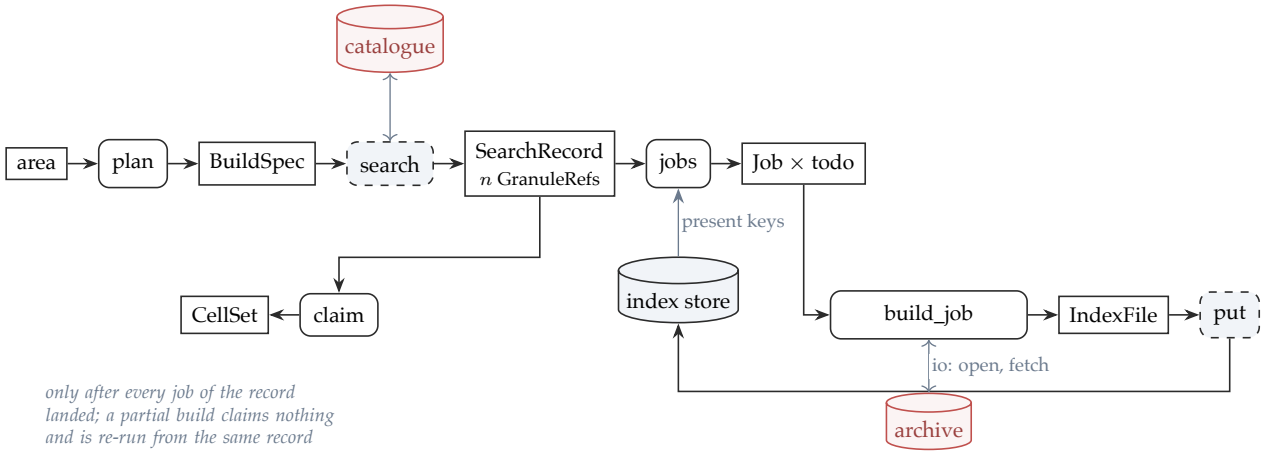


Figure 9: The build as stages over values. Rounded white boxes are pure; grey dashed boxes touch the catalogue, the archive or the disk. The record of a search is the primary value: jobs, outcomes and the claim derive from it.

Part III

The build pipeline

6 Build as a pipeline of values

A build is one sequence, whatever the collection: decide the ground, find the granules, decide which still need building, build each, record what was built. Written as stages over values, with the effects named (Figure 9):

```
plan(area, coll)                 -> BuildSpec       pure: fine cells, addressing cells, search ring
search(spec)                     -> SearchRecord    EFFECT (catalogue); stored, append-only
jobs(spec, record, present)      -> tuple[Job]      pure: refs whose file is absent or cannot prove the ground
build_job(job, coll, cellset, io)-> IndexFile       pure given io = (open, fetch): rows + footer metadata
store.put(file)                  -> key             EFFECT: tmp + rename, write-once
claim(records)                   -> CellSet         pure fold; cached as _build.json
manifest(store)                  -> Table           pure fold; cached as _coverage/manifest.parquet
```

The values are small and frozen, and they are what crosses process boundaries:

```
@dataclass(frozen=True, slots=True)
class GranuleRef:    # what a worker needs: strings, not the catalogue's object
    name: str; url: str; s3url: str; revision: str; native_id: str = ""

@dataclass(frozen=True, slots=True)
class BuildSpec:     # the ground at both resolutions; the shape the catalogue
     is asked
    collection: str; res: int; schema: str
    fine_cells: tuple[int, ...]; addressing: tuple[int, ...]
    ring: tuple[tuple[float, float], ...]; window: tuple[str, str] | None

@dataclass(frozen=True, slots=True)
class SearchRecord:  # PRIMARY, append-only: what the catalogue answered, when
    spec: BuildSpec; at: str; granules: tuple[GranuleRef, ...]

@dataclass(frozen=True, slots=True)
class Job:           # one granule under one spec; its key names the index file
    collection: str; granule: GranuleRef; res: int; schema: str; cellset_sha:
     str; params_sha: str
    @property
    def key(self) -> str: ...   # sha1(name, revision, schema, res, cellset,
     params)[:20]
```

```
@dataclass(frozen=True, slots=True)
class Outcome:
    key: str; rows: int; error: str | None
    seconds: float; requests: int; bytes: int
```

Three design points follow from writing it this way.

The record is primary. Today the catalogue's answer is cached for a day as a pickle of the client's objects and the claim is a file mutated by each build. Storing `SearchRecord`s makes the claim a fold and makes replay possible (Chapter 16): a failed build is re-run from its record, not re-searched; what a build *would* do is computed from the record and the store's keys without touching the network.

The job key names the file. With content-addressed keys the resume set is a directory listing instead of a footer read per file (33,484 reads today, before the first job starts), and the provenance question of Chapter 5 is answered by the name: a file built under a different cell set has a different key. The GPS collection stamps a metadata hash into its rows for the same reason; that is the `params_sha` slot, empty for the other five.

Pure given io. `build_job` reads a remote file, so it is not pure in the strict sense; it is pure *given* two functions, `open` and `fetch`, passed in. In production they are the archive; in tests they are a synthetic HDF5 file in memory. Nothing else in the stage touches the world, which is what makes it testable per collection without a network and what lets a worker process run it with nothing but a `Job`.

what exists today. `build.GranuleRef` (four strings plus a compatibility shim for the builders), `build.BuildCollection` (the per-collection parameters), `build.run_build` (the sequence, once, for ATL06, GEDI, GLAS, ICESSN and GPS) over `pool.run_pool`. Still proposed: `SearchRecord` stored, content-addressed keys, the claim as a fold.

7 Reading one granule

The build's cost is in `build_job`, and inside it almost entirely in reading the file's structure. Before any position is read, the HDF5 library walks the superblock, the group tree and the chunk B-trees: for an ATL06 granule about 66 small reads, each a dependent round trip, because the next address is learned from the previous read. Through a 1 MiB block cache over the object store that walk took about 2 s of a 3.3 s granule; the position fetch 0.5 s; the actual computation 0.3 s. Workers scaled the wall time until the machine ran out of threads to wait on, and then not at all (Table 2).

workers	wall (s)	granules/s	s per granule per worker	CPU busy
4	81.0	1.23	3.24	20 %
8	41.9	2.39	3.35	44 %
16	26.2	3.81	4.20	80 %
32	26.2	3.81	8.39	92 %

Table 2: ATL06, 100 granules, 4 vCPU, before the whole-object read: latency-bound to 16 workers, then flat. The per-granule time is the sum of serial round trips.

A dependent walk cannot be parallelised, but it can be made local. The granule sizes decide how (Figure 10):

- **Small granules (ATL06, GLAS, 2 to 128 MB): read the whole object once.** The first 8 MiB span is requested alone and the object's size read from the response's `Content-Range`, with no `HEAD` and no

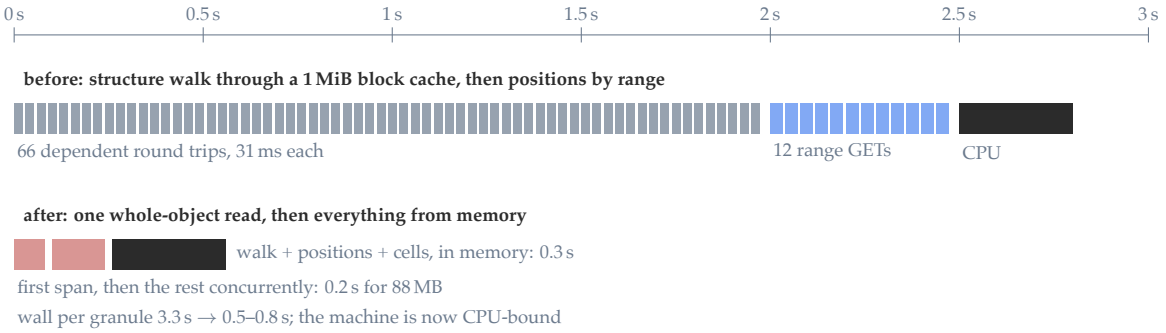


Figure 10: One ATL06 granule's build, before and after, on the same time axis. The serial walk disappears into a buffer; what remains is computation, so adding workers past the core count buys nothing and the lever becomes cores.

extra round trip; the remaining 8 MiB spans go out in one concurrent submission; the structure walk then runs over an in-memory buffer at memory speed, and the position chunks are sliced from the same buffer. Objects over a cap (256 MiB, since the per-worker peak is two to three times the object) fall back to the block cache having spent one span.

- **Large granules (ATL03, 0.2 to 6.7 GB): read bigger blocks.** The structure of an ATL03 file is clustered, so a 4 MiB block satisfies the walk in 7 reads instead of 114 while moving 29 MB instead of 120. Larger blocks read more for nothing (16 MiB: 4 reads, 67 MB, no faster).

Both cases go through one door. A `GranuleSource` is what a builder opens: `open()` returns the object the HDF5 library parses (a memory buffer, or the block-cached remote file), `fetch(ranges)` returns byte ranges from the *same* bytes (sliced from the buffer, or range GETs), and `stats` counts what was moved. Every dataset a builder reads goes through one function, `read_via_chunks`, which takes the dataset's chunk map and asks the source for the chunks, decoding them with the same decoder the query uses. The builder therefore does not know which path it is on, and a test can hand it a buffer of a synthetic file.

collection	configuration	before	after	note
ATL06, 100 granules, 8 workers	whole object	44.7 s	21.3 s	CPU 46 % → 94 %
ATL06, 300 granules, 6/8/12 workers	whole object		5.4 gran/s	flat in workers, CPU 92–98 %
ATL06, full regional rebuild (33 484)	whole object	3.9 h	62 min	9.1 gran/s sustained
GLAS, 35 granules, 8 workers	whole object	7.4 s	4.1 s	
GLAS, full rebuild (4 799)	whole object	10 min	3.5 min	
ATL03, 20 granules, 8 workers	4 MiB blocks	23.4 s	11.1 s	114 → 7 structure reads
ATL03, full regional rebuild (48 838)	4 MiB, 16 workers	12 h	≈7.3 h	1.88 gran/s, CPU-bound

Table 3: Build measurements on the deployed instance (m6i.xlarge, 4 vCPU, in-region), 2026-10-17.

The measurements (Table 3) say the build is no longer waiting on the network. What is left is computation in the worker, so the next lever is cores, not workers.

what exists today. `access.read_whole_object`, `GranuleSource`, `open_granule`; `index.read_via_chunks`; `index.ATL03_BLOCK_SIZE = 4 MiB`. ATL06 and GLAS opt in to the whole object; GEDI stays on the block cache until its sizes are measured.

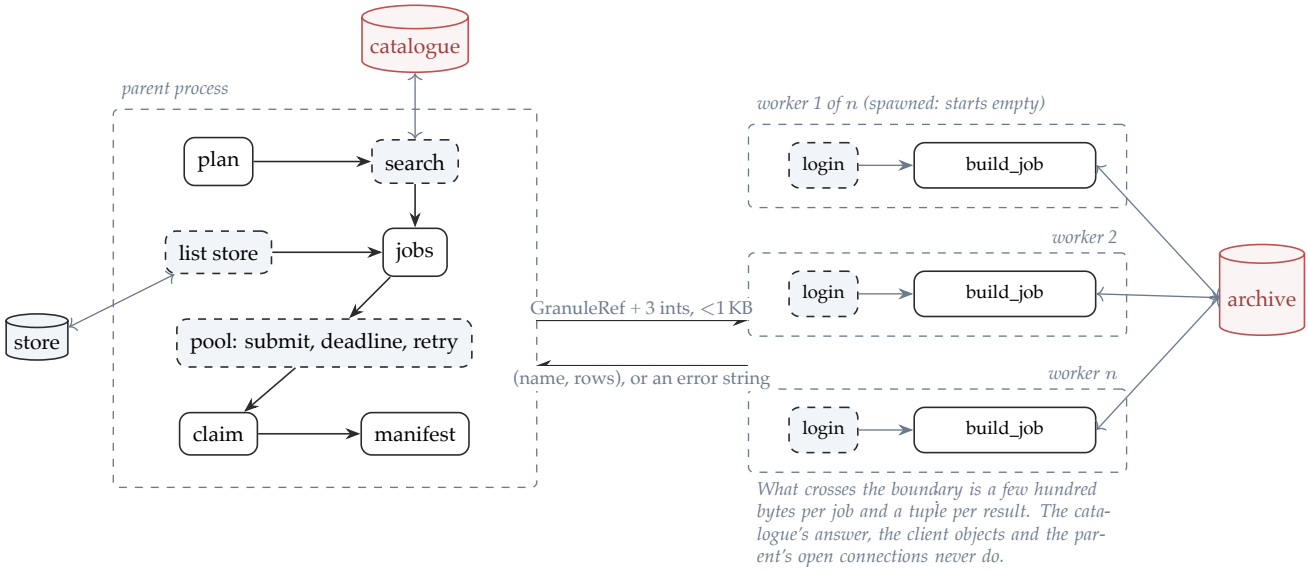


Figure 11: The build at runtime. One parent, n spawned workers, each holding its own archive client. The arrows carry values small enough to pickle without thought.

8 The build at runtime

The structural view says what is computed; this chapter says by whom. A build is one parent process and a pool of worker processes (Figure 11). The parent does everything that is cheap and must happen once: plans the ground, logs in, searches the catalogue, lists the store, decides the jobs, and at the end claims and folds the manifest. A worker does one thing: `build_job` for the job it is handed, with its own login and its own archive client.

Spawn, not fork. A forked worker begins as a copy of its parent. For a regional ATL03 build the parent holds the catalogue's answer, several gigabytes of client objects, and copy-on-write un-shares those pages worker by worker until memory approaches $(n+1)$ times the parent; a 16-worker ATL06 build was killed that way. A spawned worker begins empty and receives only its job. The price is a second of imports and a login per worker, paid once per build.

One policy for the pool. Five builders once owned five pools and disagreed on everything that matters. The one pool now has one policy: submit every job; consume the futures *in submission order* with a per-future timeout computed from a monotonic clock (a laptop that sleeps must not report a healthy build as timed out), so a progress callback sees jobs in a stable order; treat every worker exception as retryable, because the real cause of a transient archive error is destroyed in transit when the error object cannot be pickled back; run a second attempt for the failures in a fresh pool; then cancel what is left (Figure 12).

How many workers. After Chapter 7 an ATL06 granule costs about 0.7 core-seconds, so on four vCPUs the knee is at six to eight workers and sixteen buys nothing but memory. The text collections and ATL03 are still latency-bound (a whole-file GET, or a large structure walk) and take

what exists today. `pool.run_pool`, called with exactly `max_workers` and `mp_context`; `build.run_build` claims only when every job of the attempt landed and otherwise logs that it is not claiming.

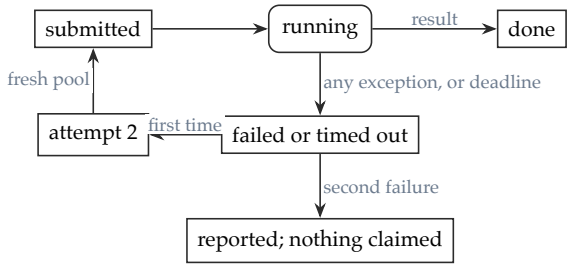


Figure 12: The life of one job in the pool. The claim at the end of the build requires every job to reach *done*; a single job in the bottom state withholds it, and the next run resumes from the files that landed.

sixteen. Memory for sixteen spawned workers with whole-object buffers peaked at 3.4 GB on a 15 GB machine; two collections are never built at once.

what exists today. `BuildCollection.default_workers`: 8 for the HDF5 builders, 16 for ATL03, ICESSN and GPS; GEDI's per-granule timeout is 600 s because a granule is a quarter orbit.

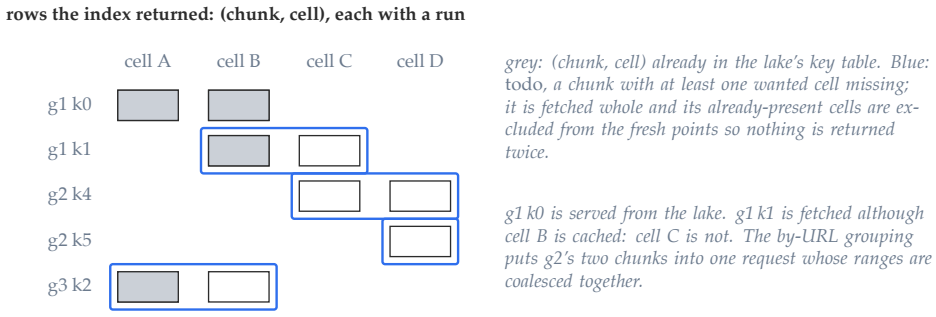


Figure 13: The plan as a set picture. The lake's key table decides *have*; a chunk goes to *todo* if any wanted cell of it is missing; todo is grouped by granule URL so one request per granule carries every range.

Part IV

The query pipeline

9 Query as a plan

A query begins as a value: the area (a box, or a polygon inside it), the time window, the collection, and a few flags (which beam groups, whether to force a re-fetch, whether to clip to cells). Turning it into points has a planning half that touches only local stores and a fetching half that touches the archive. The planning half is worth isolating because it is cheap, deterministic given the stores, and what every inspection tool (a dry run, a byte-range map, a progress denominator) wants to see.

```
cells(query, grid)                -> tuple[int]      pure: addressing cells the area overlaps
scan(query, coll, store)          -> rows            EFFECT (DuckDB over the manifest-named index files)
have(coll, granules, lake)        -> set[ChunkKey]   EFFECT (the lake's key table): what is already decoded
plan_query(query, coll, rows, have) -> QueryPlan     pure: todo = chunks with any wanted cell not in have,
                                                           completed to whole beam groups, grouped by URL
render(plan)                      -> dict            pure: what the chunk map and the progress bar show
```

The scan asks the index for every row whose cell is wanted, restricted to the beam group and window, reading only the files the manifest names. Rows arrive one per (granule, group, chunk, cell), each carrying its run. The plan groups them by chunk: a chunk's wanted cells, its runs, one representative row for its byte ranges. Against the lake's key table it splits chunks into *have* (every wanted cell already decoded) and *todo* (Figure 13). Two refinements are forced by later chapters and are decided here, not downstream:

- **A beam group is fetched whole.** If any beam of a group needs chunk k, the group's other beams' chunk k join the todo set. Chapter 11 explains why the lake's file unit makes this necessary.
- **Writes are narrowed to the wanted cells.** A chunk is far longer than a box and decodes points in cells the query never asked for. The plan records `want_only`, the cells the lake may write; the rest of the strip is discarded (Chapter 12 has the measurement that decided this).

The plan is a value and should be serialisable as one: the query, the rows

as an Arrow table, the have set and the todo list. Chapter 16 uses that to replay an ingest without the planning half.

```
@dataclass(frozen=True, slots=True)
class Query:
    collection: str; bbox: tuple[float, float, float, float]; polygon: tuple |
     None
    window: tuple[str, str] | None; res: int
    groups: tuple[str, ...] = ("strong",)       # beam groups wanted
    force: bool = False; clip_cells: bool = False

ChunkKey = tuple[str, str, int]                 # (granule, group member, chunk
     index)

@dataclass(frozen=True)
class QueryPlan:
    query: Query
    want_cells: tuple[int, ...]
    rows: pa.Table                              # one row per (chunk, cell),
     with its run
    have: frozenset[tuple[str, str, int, int]]  # (granule, beam, chunk, cell)
     in the lake
    todo: tuple[ChunkKey, ...]                  # whole beam groups
    by_url: Mapping[str, tuple[int, ...]]       # url -> row indices
    want_only: tuple[int, ...] | None
```

what exists today. `index_atl06.plan_bbox` returns this dictionary without touching the network (keys `want_cells`, `rows`, `chunk_cells`, `chunk_runs`, `chunk_row`, `have`, `todo`, `n_lake`, `by_url`, `want_only`); `_whole_groups` completes beam groups; the ATL03 planner does the same inline in `planner._ensure`.

10 Executing the plan: one chunk, step by step

Execution is the same for every todo granule and, inside it, for every chunk. This chapter follows one chunk through it and states, for each step, what the work is proportional to (Figure 14). The distinction between per-point, per-chunk and per-granule work is the whole performance story of the query side: anything per point must be native and vectorised, anything per chunk may be Python, anything per granule may be a network round trip.

Coalesce and fetch. The granule's todo rows name $5 \times m$ byte ranges. Adjacent ranges separated by less than a gap are merged into one span, because a request costs a round trip and bytes in the gap cost only bandwidth; the gap is tuned by measurement to where the two balance (about 128 KiB from outside the region, 1 MiB inside). One request per granule carries every span; granules are fetched on a thread pool sized for latency, not cores, because a thread waiting on a GET uses no CPU.

Decode. Each raw chunk is inflated, unshuffled and reinterpreted as its dtype. The three operations release the interpreter lock and run at memory speed. The decoder is the same function the build used to read positions, so a byte-identity test on one side covers the other.

Label. This is where Chapter 4 pays. For each wanted cell of the chunk the run is written into a slice of a label array; a cell absent from the runs or with an inexact run sends the whole chunk to the per-point assignment. ATL03 does the same with its edge columns and assigns only edge photons and outsiders.

Keep. One predicate, applied once to the chunk's arrays as boolean masks: the label is a wanted cell; the point lies inside the box (unless clipping to cells); the quality flag passes; and the cell is not one the lake

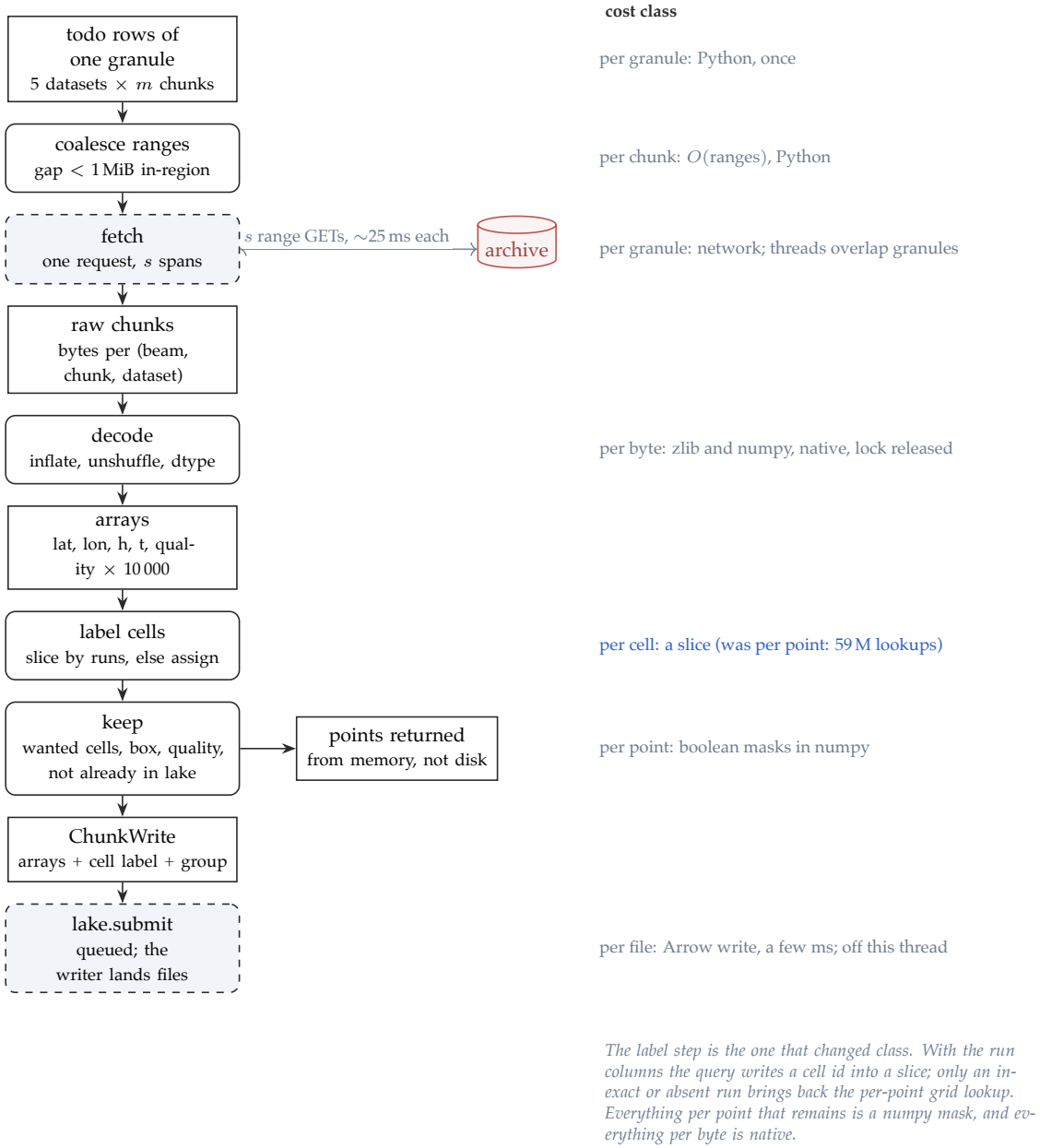


Figure 14: One chunk through the executor, with the cost class of each step. The returned points come from memory; the lake write is housekeeping for later queries and runs on other threads.

already returned for this chunk, so a partially cached chunk never contributes a cell twice. The kept points are the answer; they are returned from memory and streamed to the caller granule by granule as each fetch lands.

Hand off. The chunk's full arrays, the label and the beam group go into a `ChunkWrite` value and onto the lake's queue. The executor does not write files, wait for them or know their names. One predicate, one label, one value crossing to the writer: the five copies of this loop that exist today differ only in the per-collection decode and quality functions, which are the slots of Chapter 13.

what exists today. `index_atl06.fetch_bbox` with its inner `_ingest_granule` and `_keep`; `access.coalesce`, `decode_chunk`, `unshuffle`, `map_as_completed`; the ATL03 path in `planner._process_group`, `_decode_photons`, `_materialize`. Five copies of the loop remain (ATL06, GLAS, GEDI, ICESSN, GPS) plus the ATL03 variant.

11 The lake as a memo

The lake is a cache of decoded points, partitioned by cell, read before any fetch and written after. Losing it costs time, never truth: anything missing is re-fetched. Its design is dominated by one requirement, idempotence under concurrency, and one measurement, the cost of a file.

Layout. One Parquet file per (cell, granule, chunk, beam group), under a directory per cell:

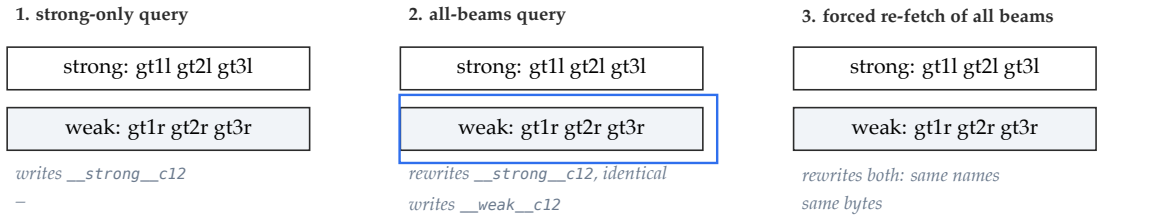
```
lake/mission=ATL06/h3_cell=599686042/<granule stem>__strong__c12.parquet
lake/mission=ATL06/h3_cell=599686042/<granule stem>__weak__c12.parquet
lake/mission=GEDI/h3_cell=.../<stem>__full__c3.parquet        full / coverage power
lake/mission=ICESSN/h3_cell=.../<stem>__na__c0.parquet        beamless collections
```

Every point goes to the file of its *own* cell, so a later query for a different sub-box of the same cell re-filters correctly; a partial cell is impossible by construction. The cell directory is the unit of reads (a query opens only the directories it asked for) and of eviction. A DuckDB table beside the lake records which (granule, beam, chunk, cell) keys have landed; it is the *have* of Chapter 9.

Why a beam group. The file count is the cost. Six files per (cell, chunk), one per beam, were 6,897 files for 2,960 chunks in one 8-cell ingest at 3 to 6 ms each, and merging chunks of one beam was measured to be a no-op because a scene box spans about one chunk per beam. Merging *beams* cuts the count, but merging all six is unsafe: the planner asks for the strong beams by default and for all six from the extraction API, so one name per (cell, chunk) would be overwritten by whichever beam set came last. The group is the unit the planner requests (`strong`, `weak`; `full`, `coverage`), so a group's file is always a full overwrite of a deterministic name with the same content, whichever request produced it (Figure 15). Six files became two: 3,225 for the same ingest.

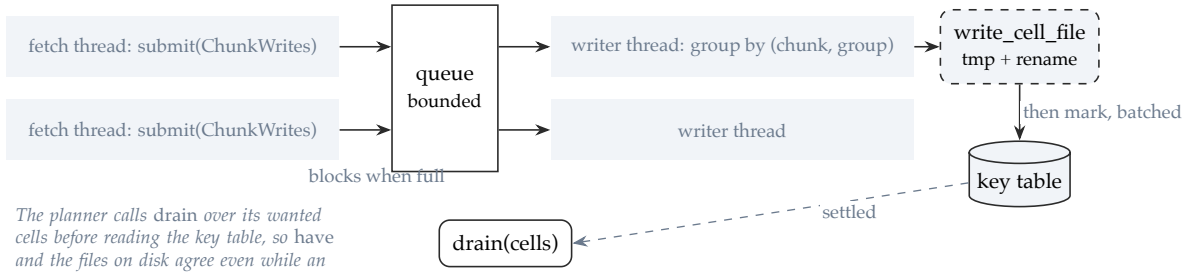
One primitive. Every lake file lands through one function: write to a temporary name in the same directory, then rename. The rename is atomic, so a concurrent reader never sees a partial file; it creates or replaces a directory entry, which is what the cell-statistics cache keys on; and it is the single place a test can block, slow or fail to exercise ordering.

The writer. Files are written by a small pool of threads fed by a bounded queue (Figure 16). The executor's threads block when the queue is full, so the fetch rate is throttled to the write rate and memory stays bounded;



Because the file is a full overwrite of a deterministic name, the planner must fetch a group whole: if one beam of a group were re-fetched alone, its file would be rewritten with that beam's points only while its siblings stayed marked as present. Chapter 9's whole-group rule is this file's correctness condition.

Figure 15: Idempotence across the two beam sets the planner can ask for. Each group's file has one name and one content; nothing merges, nothing locks, nothing lists a directory.



The planner calls drain *over its wanted cells before reading the key table, so* have *and the files on disk agree even while an earlier query is still landing files.*

Figure 16: The writer. A bounded queue gives backpressure; marking after landing gives crash safety; the drain barrier gives read-after-write consistency to the next plan.

this is why the writer is not made "pure" by handing writes back to the caller as values, which would lose the backpressure. Two ordering rules are the contract: a key is marked present only *after* its file is on disk, so a crash loses cache and never truth; and a barrier exists that waits until every queued write over a set of cells is on disk and marked, which the planner calls before reading *have*, so a second query over the same area never reads a half-written lake. Marks are batched into one database transaction per flush because opening the key table costs 150 ms.

Failure unit. A group's chunk may touch several cells, and one cell's file may fail. The write returns, per cell, whether its file landed; the writer marks every beam of the group present in the cells that landed and in none of the cells that failed, so the chunk is re-fetched there rather than believed. Cells with no valid points are marked too, or an all-fill cell would be re-fetched forever.

Eviction. The lake has a byte budget per mission and globally. Eviction is by cell directory, least recently used, protecting the cells of the query in flight, and runs off the request thread. The cell statistics it needs are cached per directory and invalidated by the directory's modification time, which the rename updates.

12 The query at runtime

Four kinds of thread and one kind of process take part in a query (Figure 17). The request thread plans, reads the lake, and streams results to

what exists today. `lake.write_cell_file`, `write_chunk_group` (returns `{cell: landed}`), `ChunkWrite.group`, `_Writer` with `submit`/`drain`/`_flush`, `mark_ingested_many`, `ingested_chunk_cells`; `meta.duckdb` beside the index. ATL03 photons still write inline from their worker processes via `write_photons`.

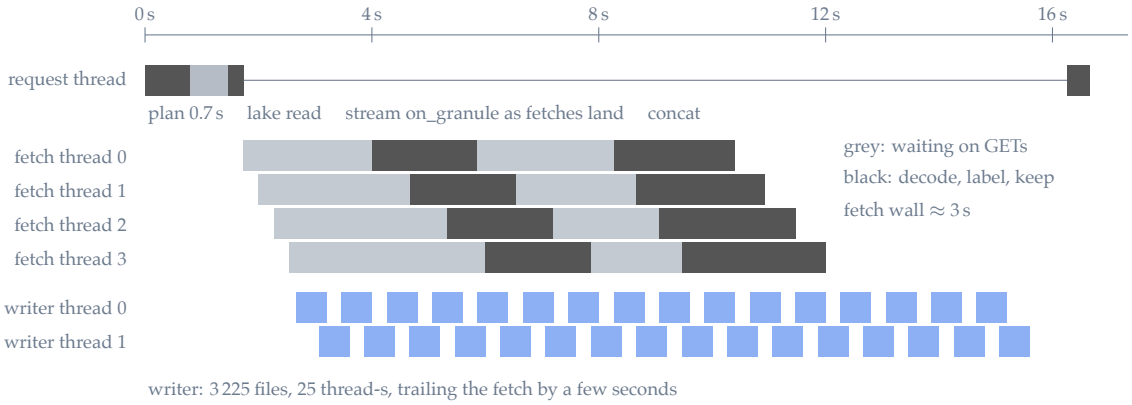


Figure 17: One query's threads over time. Fetch threads alternate between waiting and decoding; the writer trails; the request thread streams each granule's points as its fetch lands rather than after the slowest.

the caller as they arrive. A pool of fetch threads runs the executor of Chapter 10, one granule per task; it is sized for latency (up to 16) because its threads wait. The writer threads land files. For ATL03 only, decoding and the co-registration of 100,000 photons per chunk is CPU work heavy enough to be sent to spawned worker processes, one per (granule, beam) group, which fetch, decode, assign and write their own files and return the key set for the parent to mark.

Where the time goes, and what it rules out. Fetch wall is about 3 s for a scene: the archive client sustains 670 to 840 MB/s in-region and the request count is set by coalescing. The rest is CPU: decode, the remaining per-point masks, Arrow writes. The ratio of CPU-seconds to wall-seconds is 2.45 on two physical cores, which answers the question that always comes next. A process pool for decode and write would only help if the interpreter lock were serialising the threads, which would show as a ratio near 1.0 with wall well above fetch; at 2.45 the threads already overlap and the machine is simply out of cores. The ATL03 process pool is justified by its heavier per-chunk CPU, and it is spawned, not forked, because it runs inside a web server that holds a live archive client, an event loop and the writer's threads, none of which survive a fork intact.

What was measured not to help. Pre-caching the cells adjacent to the request (writing the whole decoded strip rather than the wanted cells) was tested twice. It made the first query 1.9 to 2.1 times slower and the neighbouring query slower too, because a chunk is re-fetched if *any* wanted cell is missing and the strip only brushes the neighbour. Raising the archive client's connection pool did nothing: the throughput cap is the machine's. Both results are recorded where the knobs live so the hypotheses are not re-tested by accident.

what exists today. `access.map_as_completed` (completion-order streaming), `FETCH_WORKER_CAP = 16`, `lake._Writer`, `planner._ensure` with a spawn `ProcessPoolExecutor`; the measurements in the `precache_adjacent` and `open_granule` docstrings and in `scripts/bench_ingest_phases.py`.

Part V

The framework

13 The collection descriptor

Parts III and IV described one build and one query. Today there are six of each, and they differ in exactly the ways Table 1 listed: how a file's structure maps to element intervals, where positions come from, how a chunk is decoded into points, what quality means, which group a beam belongs to. Everything else (the driver, the pool, the resume set, the claim, the plan, the executor's loop, the keep predicate, the writer) is the same sequence written several times with those differences inlined. The framework is the step of making the differences a *value*, the `Collection` descriptor, and the sequence a single implementation that reads the value where it needs to (Figure 18).

```
@dataclass(frozen=True, slots=True)
class Dataset:
    key: str            # index-row and points column name
    path: str           # HDF5 path relative to the group, or a CSV field index
    role: str = ""      # "lat" | "lon" | "h" | "t" | "quality" | ""

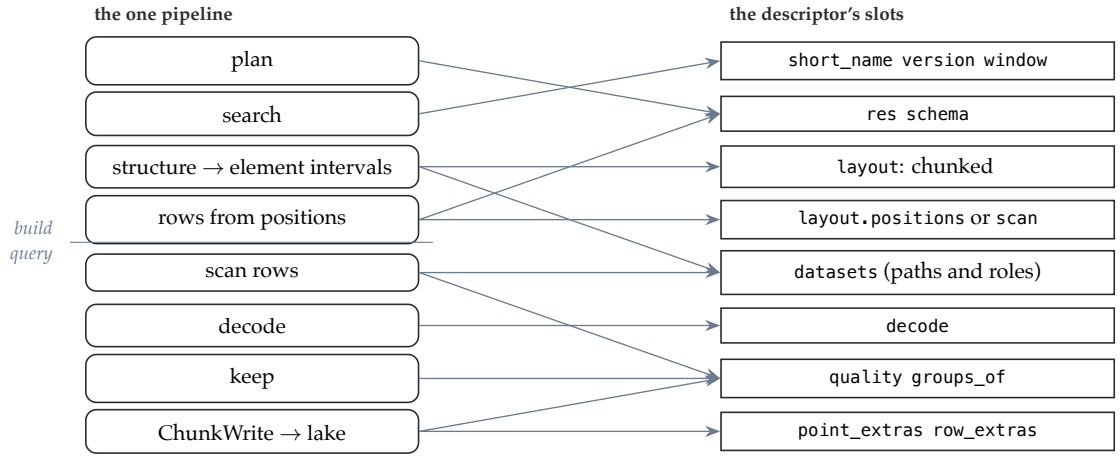
@dataclass(frozen=True, slots=True)
class HDF5Chunked:      # ATL03, ATL06, GLAS: one chunk map per group
    groups: tuple[str, ...]          # beams; ("",) for GLAS
    reference: str                   # the dataset whose chunking is the group's
    positions: Callable              # (group, datasets, fetch) -> Positions
    group_scalars: Callable | None   # (file) -> {"sc_orient", "sdp_epoch"}

@dataclass(frozen=True, slots=True)
class HDF5Spans:        # GEDI: rows are spans at the union of boundaries
    groups: tuple[str, ...]; positions: Callable; quality_field: Callable

@dataclass(frozen=True, slots=True)
class TextLines:        # ICESSN, GPS: rows are line byte spans per cell
    scan: Callable      # (bytes) -> LineScan(lat, lon, start, end)
    header_in_rows: bool

@dataclass(frozen=True, slots=True)
class Collection:
    key: str; mission: str; short_name: str; version: str
    res: int; schema: str
    window: tuple[str, str] | None
    layout: HDF5Chunked | HDF5Spans | TextLines
    datasets: tuple[Dataset, ...]
    parse_name: Callable[[str], dict]           # orbit numbers and dates from
     the name
    date_of: Callable[[str, dict], str]         # "YYYYMMDD" for window and
     manifest
    decode: Callable                            # (raws, row) -> Points, pre-
     mask
    quality: Callable | None                    # (Points) -> bool mask
    groups_of: Callable[[dict], Mapping[str, str]]   # beam -> "strong" | "weak"
      ...
    row_extras: tuple[str, ...] = ()            # extra index columns (GPS
     geometry)
    point_extras: tuple[str, ...] = ()          # extra lake columns ("quality",
      ...)
    job_params: Callable | None = None          # (record, io) -> (params, sha);
      GPS
```

Every function-valued field is a module-level function, so the descriptor



An edge means "this stage reads this slot". The stages on the left exist once. Adding a collection adds a column of values on the right and, at most, two small functions (`positions` or `scan`, and `decode`). No driver, no ingest loop, no resume set.

Figure 18: The plug board: which stage of the one pipeline consults which slot of a collection's descriptor. This is the structural meaning of "pluggable".

pickles across a spawn boundary as a name and a worker looks it up in a registry. Presentation (labels, colours, default selection) is not in the descriptor; it is an application table keyed by `Collection.key`, because the framework never draws anything.

Two descriptors, written out. ATL06, the reference HDF5 collection:

```
ATL06 = Collection(
    key="ATL06", mission="ATL06", short_name="ATL06", version="007", res=5,
     schema="atl06-3",
    window=("2018-10-01", "2027-01-01"),
    layout=HDF5Chunked(groups=("gt1l","gt1r","gt2l","gt2r","gt3l","gt3r"),
     reference="latitude",
        positions=slots.positions_identity,      # lat/lon per element
        group_scalars=slots.icesat2_scalars),    # sc_orient, sdp epoch
    datasets=(Dataset("latitude", "land_ice_segments/latitude", "lat"),
              Dataset("longitude", "land_ice_segments/longitude", "lon"),
              Dataset("h_li", "land_ice_segments/h_li", "h"),
              Dataset("delta_time", "land_ice_segments/delta_time", "t"),
              Dataset("atl06_quality_summary", "land_ice_segments/
     atl06_quality_summary",
                      "quality")),
    parse_name=slots.parse_icesat2_name, date_of=slots.date_from_name,
    decode=slots.decode_shared_chunk, quality=slots.quality_equals_zero,
    groups_of=slots.icesat2_strong_weak, point_extras=("quality",),
)
```

ICESSN, a text collection:

```
ICESSN = Collection(
    key="ICESSN", mission="ICESSN", short_name="ILATM2", version="2", res=5,
     schema="icessn-4",
    window=None, layout=TextLines(scan=icessn.scan_lines, header_in_rows=False),
    datasets=(), parse_name=icessn.parse_name, date_of=slots.date_from_name,
    row_extras=("itrf_year",), decode=icessn.parse_span_points, quality=None,
    groups_of=slots.no_groups,
    point_extras=("sn_slope", "we_slope", "itrf_year"),
)
```

GPS differs from ICESSN in four fields: `header_in_rows=True`, five row extras carrying the sled geometry, a `job_params` that reads the traverse-metadata granule once in the parent and hashes it, and its own `decode`.

The same pattern that stamps a parameter hash into a job key (Chapter 6) is what makes a metadata change invalidate the right files.

Where today's code goes. The per-collection functions that exist today become the slots; nothing is rewritten, only moved and given a signature (Table 4).

slot	existing code
`positions` (ATL03)	the segment-to-photon mapping in the ATL03 builder
`positions_identity`	ATL06 and GLAS position reads via `read_via_chunks`
`HDF5Spans` cutting	`index_gedi.chunk_spans`, `chunk_of`, `span_offset` (already pure)
`TextLines.scan`	`index_icessn._parse_fields` and its loop; the GPS equivalents
`decode`	`index_atl06._decode_chunk`, `index_glas._decode_chunk`, `index_gedi._decode_span`, `planner._decode_photons`, the two CSV parsers
`quality`	the `quality==0` masks, `index_gedi.quality_field`, GLAS's saturation cap
`groups_of`	`index.strong_beams` and GEDI's power flag
`job_params`	`build._gps_prepare`, `resolve_geometry`
row emission	the per-(chunk, cell) loops in every builder, written once

Table 4: Existing functions that become descriptor slots.

what exists today. `build.BuildCollection` is the build half of this descriptor (key, short name, version, resolution, index directory, resume set, timeout, workers, prepare); `coverage.collections()` holds the presentation half. The query half and the layout kinds are proposed.

14 Configuration as a value

The pipeline has about two dozen tunables: fetch mechanism, coalescing gap, thread and worker counts, lake writers, byte budgets, data directory, token location. Today they are read from the environment at about thirty sites, some at import time, which means a test, a bench and a worker process each discover the configuration by accident. The design makes configuration a frozen value built once at each *door* of the process and passed down (Figure 19).

```
@dataclass(frozen=True, slots=True)
class Config:
    data_dir: Path
    in_region: bool; s3_mechanism: str; coalesce_gap: int; crt_gbps: int
    fetch_workers: int; fetch_threads: int
    index_workers: int; job_timeout_s: int; start_method: str = "spawn"
    whole_object_max_bytes: int = 256 << 20
    lake_writers: int; lake_async: bool; lake_max_bytes: int
    precache_adjacent: bool
    token: str | None; edl_file: Path
    per_collection: Mapping[str, Overrides]   # e.g. GEDI job_timeout_s=600
    now: Callable[[], datetime] = utcnow                  # an injected clock

Config.from_sources(env, cli, file) -> Config   # cli > env > file > defaults
```

The environment variable names stay as they are, so deployment files are unchanged; what changes is that below the door nothing reads them. Customisation is by argument, not by subclass or monkeypatch: a different lake layout is a different `Layout` value passed to the lake; a different coalescing policy is a different function passed to the fetcher; a test's fake archive is a `fetcher` argument to the executor. The one transitional concession is a process-wide accessor returning the config set at the door, which old modules call until they are converted, and which an AST test forbids in the new packages.

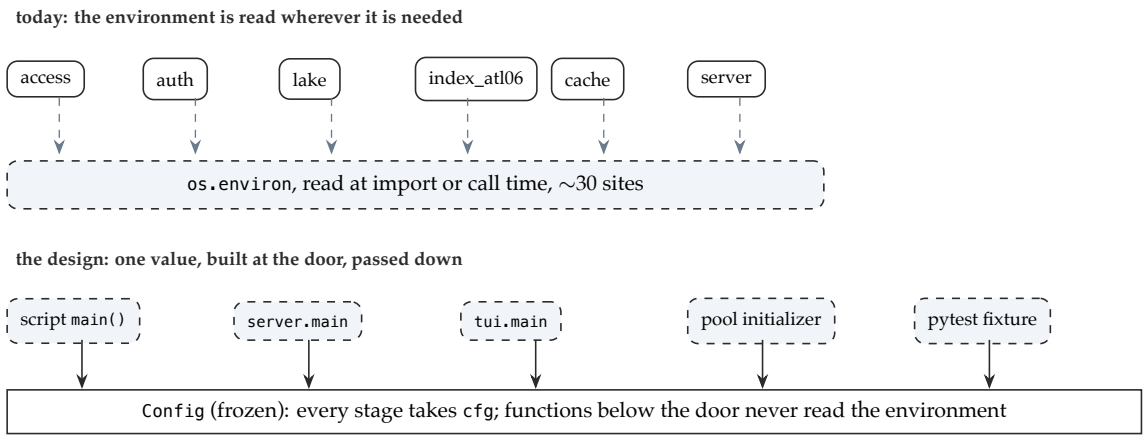


Figure 19: Configuration moves from scattered environment reads to one value built at each process door. A transitional process-wide accessor lets old modules read it until they take `cfg` explicitly.

15 The grid as a parameter

Chapter 3 introduced cells through H3 and promised to say what the design needs from a grid. The honest list is short, and the performance contract decides what may be abstracted at all.

```
@dataclass(frozen=True, slots=True)
class Grid:                          # one value per implementation
    name: str                        # "h3" | "s2" | "rhealpix"
    resolutions: range
    convex_cells: bool               # may the exact-run shortcut be trusted?
    cell_area_km2: Callable[[int], float]
    cells_vectorized: Callable[[Arr, Arr, int], Arr]       # lat, lon -> ids:
     THE hot kernel
    cells_for_area: Callable[[Ring, int], Sequence[int]]   # polyfill, overlap
     semantics
    parent: Callable[[int, int], int]
    children: Callable[[int, int], Sequence[int]]
    compact: Callable[[Sequence[int]], Sequence[int]]      # the claim's
     representation
    boundary: Callable[[int], Sequence[tuple[float, float]]]
    neighbors: Callable[[int, int], Sequence[int]]
    to_text: Callable[[int], str]; from_text: Callable[[str], int]
```

Three rules follow (Figure 20). The per-point kernel must be native; a grid without one is rejected, not tolerated. Per-cell operations run hundreds of times per query and may be plain Python. Storage ids are 64-bit integers whatever the grid's native form, because the index scan, the lake partitions and the manifest all rely on a sortable scalar; a grid whose ids are strings (rHEALPix's `"N2345"`) packs them into 64 bits in its module and exposes the text form for pages and the API.

The exact-run shortcut of Chapter 4 rests on one geometric fact, that a straight track meets a cell in one contiguous stretch, which holds for convex cells. H3 hexagons, S2 cells and rHEALPix cells are convex; a grid that is not sets `convex_cells` false, which disables the fast path and keeps the per-point fallback.

The grid is part of the store's identity: an index and a lake built under one grid are never read under another, so the grid's name is recorded



Figure 20: The three cost tiers of grid operations. Only the innermost is performance-critical, and it is the one operation a grid implementation must provide natively; a descriptor validates it against a time budget on 10^5 points at import.

beside them and carried in job keys and plans. Column and partition names drop the grid's name (`h3_cell` becomes `cell`), a one-time rename that rides with a schema bump. The addressing resolution becomes per grid in the descriptor, chosen by the same rule as today, a cell about the footprint of a chunk: H3 5, S2 9, rHEALPix 4 for ATL06.[6]

[6] Adding S2 is a module plus the C++ bindings' vectorised kernel. rHEALPix has no vectorised kernel in the ecosystem; it is the case that would justify a small Rust extension, which this structure makes a 200-line package rather than a change to the framework.

16 Replay

Three values make the system replayable, and each pays for itself in a different place.



`SearchRecord`: the job list recomputed without the catalogue. `QueryPlan`: an ingest re-executed without planning. Recorded bytes: decode and planner tests on real chunks without the archive.

Figure 21: Record once, replay without the effect.

- **Build.** A stored `SearchRecord` plus the current descriptor and the store's keys recomputes the job list exactly. A failed build is re-run from its record; a staleness policy on the record's timestamp decides when to search again. The build map that an explainer page reconstructs by hand becomes `render(record)`.
- **Query.** A `QueryPlan` serialises to a directory (the query as JSON, the rows as Arrow) and `execute(load(plan))` replays it. The ingest benchmark becomes "record once, execute N times with a different writer count", which is the honest form of the measurement Chapter 12 relied on.
- **Bytes.** A recording fetcher wraps any fetcher and writes each (url, offset, size) request's bytes to a fixture directory. One small granule per collection, recorded once in-region, lets decode and planner tests run on real bytes instead of synthetic blobs.

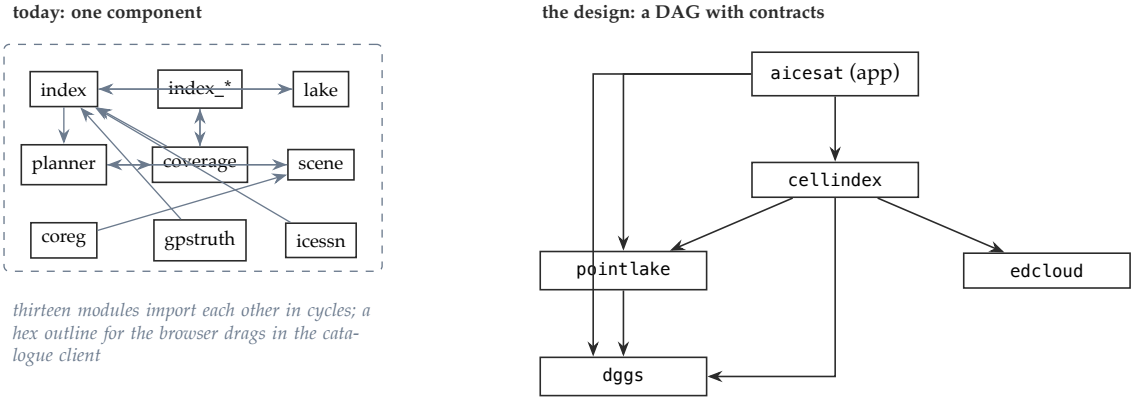


Figure 22: Before and after. The cycles of today's single component become a four-package DAG under the application, with layer contracts inside the framework package.

17 Packages and the structural view

The code today is one package of 13,000 lines whose index, lake, planner, coverage and collection modules form a single strongly connected import component of thirteen modules. The design splits it along the two seams the application crosses without the framework (grid geometry and the point cache) and keeps the rest as layers inside one framework package, separated by import contracts rather than by packaging (Figure 22).

package	lines	responsibility	who else would use it
`dggs`	∼400	grid geometry over areas and cell sets; `Grid` values for H3 (and later S2)	anyone indexing by a global grid
`edcloud`	∼900	Earthdata mechanisms: login, catalogue search to `SearchRecord`, range and whole-object GETs, HDF5 chunk maps, decode	any project reading archive granules by byte range
`pointlake`	∼900	cell-partitioned Parquet memo, key table, writer, eviction	any cell-keyed point cache
`cellindex`	∼2 500	the framework: descriptor, values, build and query pipelines, store, claim, manifest	another altimetry or lidar project with its own descriptor
`aicesat`	the rest	scenes, analysis, API, server, TUI, browser, deploy	not reused

Table 5: The packages. One repository, one uv workspace; separate repositories only when an external consumer needs a different release cadence.

The mapping from today's modules is direct: `access` becomes `edcloud` (with the pure parts, coalescing and decode, separated from the mechanisms); `lake` becomes `pointlake`; `planner`'s geometry becomes `dggs`; `index`, `index_*`, `build`, `pool`, the rest of `planner` and the non-presentation half of `coverage` become `cellindex`; the analysis modules (co-registration, time series, lake levels, DEM and imagery readers) stay in the application, already pure at the function level, until a second consumer appears.

18 Purity in Python, and what not to do

Python enforces none of the above. The design is kept honest by tools that run in the one check target, since there is no continuous integration today:

1. **Import contracts** (import-linter): the layer order inside `cellindex` and the forbidden imports listed under Figure 22; the application may not import the build runner from request handlers.
2. **An AST purity test** over the pure module list, failing on environment reads, file opens, clocks, randomness, caches, threading and any module-level mutable assignment. The test that today forbids the query path from reaching the catalogue is this test's ancestor and is already parsed, not grepped.
3. **Types**: strict checking on the value modules and the grid package from day one, ratcheted elsewhere.
4. **Lint rules** that catch the usual leaks: naive clocks, prints in libraries, mutable defaults.
5. **Frozen, slotted values** with validation in `__post_init__` rather than defensive code downstream.
6. **Coverage floors** per package, and the golden suite (nine numbers against a cloned data directory) as the behavioural floor for the application at every migration step.

And the list of things the design deliberately does not do, each of which was considered:

- No effect system. Values are used in the four places replay or parallelism pays (`SearchRecord`, `Job`, `QueryPlan`, `ChunkWrite`); everywhere else a function that takes `cfg` and `io` arguments is the whole pattern.
- No class hierarchy and no plugin entry points for collections. A frozen dataclass with function-valued fields, one module per collection, pickles across spawn and is all that is needed.
- No storage interfaces over DuckDB, Parquet or the grid kernel. They *are* the performance; the pluggable points are the fetch mechanism, the layout value, the cell kernel and the coalescing policy, each a plain argument.
- No repository split, package index or independent versioning before a second consumer exists.
- No "pure" writer. The queue's backpressure is what bounds memory.
- No refactoring of the leaf kernels (decode, coalesce, span cutting, the parsers). They are the parts that are already right.
- No trust in purity by convention. Hence the list above.

Appendices

A Migration path

Each step lands on the main branch, passes the full suite and the golden suite, and deploys. Steps are ordered so that each removes a hazard on its own.

step	what	retires	effort	status
A	`Config` at the doors; `from_sources`; `cfg` threaded through the fetcher, lake and planner; transitional accessor	environment reads below the doors	2 d	
B	`GranuleRef` and `Job`; one `run_build`; spawn pool with deadline and retry; resolution guard in every builder	five drivers, five fork pools, per-task pickling of the cell set	3 d	partly landed 2026-10-17; `SearchRecord` and the pool initializer remain
C	`SearchRecord` stored; the claim as a fold writing today's file shape	claim mutation, claim invalidation, the catalogue pickle	2 d	
D	content-addressed index store; manifest rows carry keys; glob fallbacks deleted; explicit conservative GC; one-shot rename on the box	six resume sets, footer provenance reads, the sidecars (the sha is in the key)	4 d	highest risk: 120k files renamed; rehearse on the clone
E	`Collection` descriptor and one `build_job` with three layouts; `Grid` enters here	six builder bodies, per-collection version constants	3 d	
F	`QueryPlan`, one `execute`, one `keep`; the lake takes a cell column	five fetch loops, five keep predicates, the ATL03 variant	5 d	seeds landed: one cell pass, group files, runs
G	extract packages; uv workspace; import contracts; delete the accessor	module singletons as cross-package state	3 d	
H	application split: scenes, analysis, admin; presentation registry; TUI over the public API	duplicated mission tables	4 d	

Table 6: About 26 days of focused work; D and F carry the least trustworthy estimates.

B Test strategy

package	pure-core tests	interpreter tests	property tests	floor
`dggs`	literal geometry; claim compaction	none	compact and membership preserve every cell; polyfill covers centre-contained cells; the kernel agrees with the scalar API and meets the time budget	100 %
`edcloud`	coalesce, decode, span cutting on literals and recorded blobs	mechanisms behind a fake archive; catalogue with fixtures; token parsing	coalesce covers every input and respects the gap; decode∘encode identity with shuffle and gzip	95 / 80 %
`pointlake`	cell tables with a cell column; dtype preservation	writer ordering, eviction, stats, query scoping under a temporary config	write then query returns the same multiset	90 %
`cellindex`	jobs from literal records; rows from positions; plan on Arrow literals (todo ∩ have = ∅, every wanted cell is in lake or todo); keep; claim fold	`build_job` with a fake io (in-memory HDF5 plus recorded blobs); `run_build` with an inline pool; `execute` with a fake fetcher, byte-identical to the direct fetch for all six collections	job key stable under field order; plan idempotent	95 / 85 %
`aicesat`	science modules as they are	API, server, stream, TUI as they are; golden suite unchanged at every step		85 %

Table 7: Per-package test shape and coverage floors. Of today's ∼10 500 test lines, about 1 900 move with their modules, about 1 200 retire with what they tested, and the rest stay in the application.

C Measurements the design leans on

measurement	before	after	instrument
ATL06 build, 100 granules, 8 workers	44.7 s	21.3 s	`bench_build_io.py`
ATL06 build, workers 4 / 8 / 16 / 32 (before whole object)	81 / 42 / 26 / 26 s		`bench_build_io.py`
ATL06 per granule, serial reads through the block cache	66 reads, 2.0 s		profile
ATL06 full regional rebuild, 33 484 granules	3.9 h	62 min	staging rebuild
GLAS full rebuild, 4 799 granules	10 min	3.5 min	staging rebuild
ATL03 structure reads per granule at 1 / 4 / 16 MiB blocks	114 / 7 / 4		`bench_build_io.py`
ATL03 full regional rebuild, 48 838 granules, 16 workers	12 h	≈7.3 h	staging rebuild
16 spawned workers, peak memory		3.4 GB	`free -m`
8-cell ATL06 ingest, wall	25.4 s	16.6 s	`bench_ingest_fresh.py`
same, CPU-seconds	40.7	32.4	idem
same, CPU-s / wall-s (2 physical cores)		2.45	idem
same, lake files written	6 897	3 225	idem
same, cell assignment thread-seconds	24–31	0	idem; 2 948 of 2 948 chunks from runs
exact runs on real ATL06 rows (12 granules)		99.3 %	`tests/test_index_cell_runs.py`
ATL03 plan for a 0.3° box	36 s	0.66 s	query trace
index store with sidecar provenance	7.5 GB	4.7 GB	`migrate_index_footers.py`
in-region fetch throughput, CRT client	300–400	670–840 MB/s	`bench_coalesce.py`
precache-adjacent, first box / neighbour (on vs off)	16.8 / 13.9 s	31.9 / 21.7 s	`bench_ingest_fresh.py`

Table 8: Measurements on the deployed instance (m6i.xlarge, 4 vCPU, 15 GB, us-west-2), 2026-10-16 and 17.

D Glossary

granule one archive file: an orbit segment, a flight, a survey.

chunk a fixed-size, separately compressed piece of an HDF5 dataset; the unit of a byte range.

chunk map (offset, size, filters) for every chunk of a dataset, read from the file's B-tree once at build.

span GEDI's row unit: an element interval inside one chunk of every dataset.

cell a grid cell at some resolution; addressing cells key rows and lake files, claim cells record coverage.

run the element interval of one cell inside one chunk, and whether it is pure.

index one Parquet file per granule of (chunk, cell) rows carrying byte ranges and runs.

claim the compacted cell set a collection's index is known to cover; consulted before every query.

manifest the fold of an index into distinct (cell, granule, month) rows; names the files a query opens.

provenance the filter set a granule's file was built against, recorded in its footer by hash.

lake the cell-partitioned Parquet memo of decoded points, read before and written after a fetch.

beam group the set of beams the planner requests together and the lake files together.

have, todo the chunks a plan serves from the lake and the chunks it fetches.

door a process entry point where configuration is built once: a script's main, the server, the TUI, a pool initializer, a test fixture.

slot a function-valued field of a collection descriptor that a pipeline stage calls.

value immutable data; a frozen dataclass here.

effect a function that touches the network, the disk, a clock or a pool.